

第6章 优先队列(堆)

建议学时:4-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 理解**优先队列** ADT(insert/deleteMin/findMin)与第3章普通队列的区别:出队按优先级而非到达顺序
- 掌握二叉堆的**结构性性质**(完全二叉树)与**堆序性质**(父 $\leq$ 子)
- 掌握 0 基数组表示(根=0、左子 $2i+1$ 、右子 $2i+2$ 、父 $(i-1)//2$),能默写**上滤**与**下滤**
- 能解释两种建堆方式的复杂度差异
- 能说出 d-堆、左式堆、斜堆、二项队列的动机与差别
- 会用 `heapq` 四函数(heapify/heappush/heappop/heapreplace),掌握大顶堆的负号技巧
- 能用"大小为 k 的小顶堆"独立解决 top-k,并用 `heapq` 验证手写堆

💡 从 C 到 Python:本章主题如何迁移

优先队列的直觉一句话: "不用排队,按重要性插队"。第3章的普通队列是"先来先服务",而急诊分诊、任务调度、短作业打印都要求"最紧急者先处理"。它只有三个操作:insert、findMin、deleteMin。

在 C 里,朴素做法两头为难:无序数组 insert $O(1)$ 但 deleteMin 要扫描 $O(n)$;有序数组反过来了。二叉堆的巧妙之处在于 insert 与 deleteMin 同时做到 $O(\log n)$,且不需要指针——完全二叉树直接摊平进数组,父子关系用下标计算,正是 C 程序员最熟悉的数组思维。

Python 迁移分三层:底层用 `list` (就是动态数组)当堆数组;中间层手写二叉堆类(\$6.2-\$6.4);上层用标准库 `heapq` (\$6.6)。注意 C++ 教材用 1 基(下标 0 闲置),Python 的 `heapq` 用 0 基(\$6.2.3)。

📦 前置知识自查

数学前置:完全二叉树高度 $\approx \log_2 n$;求和记号 Σ 与几何级数(建堆 $O(n)$ 推导用);摊还分析直觉(第11章)。

C 语言前置:数组与下标运算; `struct` 与函数指针(对照用);整数除法(0 基父结点 $(i-1)//2$,与 C 的负数除法不同)。

依赖前面章节:第2章大 O;第3章 `list` (堆的底层数组)与栈;第4章树术语(父、子、叶、深度、高度)。

🚀 本章学习路线

- 第一阶段(1 小时):**读 \$6.1-\$6.2,理解 ADT 与两条性质,背熟 0 基公式
- 第二阶段(1.5 小时):**读 \$6.3-\$6.4,手写上滤/下滤/建堆代码,手工走 insert 与 deleteMin
- 第三阶段(1 小时):**读 \$6.5-\$6.6,俯瞰堆家族,上机实验 `heapq` 四函数与负号技巧
- 第四阶段(实验,1 小时):**完成章末实验:Top-K 高频词
- 验收标准:**能默写上滤与下滤,说出"二叉堆 insert/deleteMin $O(\log n)$ 、建堆 $O(n)$ ",解释 `heapq` 为什么默认是小顶堆

本章知识导图

- §6.1 优先队列模型与动机(任务排程/急诊分诊、ADT 三操作、队列对比)
- §6.2 二叉堆:结构性质与数组表示(完全二叉树、堆序性质、0 基 vs 1 基换算)
- §6.3 插入与删除最小:上滤与下滤(完整代码、"末元素搬根再下滤"套路)
- §6.4 建堆与其余操作(逐个插入 $O(n \log n)$ vs 下滤建堆 $O(n)$ 、decreaseKey/delete/merge)
- §6.5 其他堆:d-堆、左式堆、斜堆与二项队列(动机与一句话差别)
- §6.6 标准库:heapq(默认小顶堆、heapify/heappush/heappop/heapreplace、负号技巧)
- §6.7 手写堆与标准库并排:top-k 实战(小顶堆淘汰技巧、堆排序预告)

§6.1 优先队列模型与动机

6.1.1 三个场景:谁最紧急谁先走

先看三个场景。① **操作系统任务调度**:每次选择优先级最高的进程;② **打印机队列**:短作业优先;③ **急诊分诊**:按病情严重程度处理,最危重者最先抢救。

三个场景的共同点一致:插入一个元素、查看当前最紧急的元素、取出该元素。大小关系代表优先级还是病情,由具体问题映射,数据结构不管。

6.1.2 优先队列 ADT:三个操作

优先队列(priority queue) 是抽象数据类型: `insert(x)` 插入; `findMin()` 返回最小元但不删除; `deleteMin()` 删除并返回最小元。约定数值越小优先级越高(最小堆),取最大则把比较方向反过来(§6.6 负号技巧)。注意:**堆(heap)** 是优先队列最主流的实现,不是 ADT 本身;与存放动态内存的"堆内存"同名,毫无关系。

记号说明(本章约定,与 C++ 版一致)

复杂度记号沿用第2章。下标公式中,"父 $(i-1)//2$ "与"父 $i/2$ "都指**整数除法**(代码写 $(i-1)//2$ 、 $i//2$)。"最小元""最高优先级"同义。

6.1.3 与普通队列对比:出队顺序由谁决定

第3章的普通队列是 FIFO:出队顺序 = 到达顺序;优先队列相反:出队顺序 = 优先级顺序。朴素实现却两头为难:

朴素实现	insert	deleteMin	问题
无序数组/链表	$O(1)$	$O(n)$ (扫描)	删除太慢
有序数组/链表	$O(n)$ (移位)	$O(1)$	插入太慢
二叉堆(本章主角)	$O(\log n)$	$O(\log n)$	两者均衡

C 的写法	Python 的写法
<code>struct PQ { int *a; int n; };</code> 数据与操作分离	<code>class MinHeap:</code> 数据与操作封装在类里
<code>pq_deleteMin(&pq, &out);</code> 每个操作传指针	<code>h.pop()</code> 方法天然绑定对象
找最小要自己写 <code>for</code> 循环扫描	根就是最小元, <code>h.top()</code> 一行
数组长度、容量、释放全靠手工	<code>list</code> 自动扩容, 无指针无释放

§6.2 二叉堆:结构性质与数组表示

6.2.1 结构性质:完全二叉树

二叉堆首先是一棵**完全二叉树**: 除最后一层外每层都满, 最后一层从左到右连续填充、不留空隙。这条性质的威力在高度: 结点数 n 满足 $2^h \leq n < 2^{h+1}$, 所以 $h \approx \log_2 n$ ——任何根到叶路径只有对数长度, 这正是上滤/下滤高效的根源。

完全二叉树还不需要指针: 结点按层序编号, 父子关系用下标直接算出 (§6.2.3)。对比第4章的指针树, 连续数组节省指针空间、缓存友好, 代价是形状被锁定, 只能在末尾增删。

6.2.2 堆序性质:最小堆

堆序性质: 对每个非叶结点, 父结点的值 $\leq$ 子结点的值 (最小堆; 换成 $\geq$ 即最大堆)。两条性质合起来才是二叉堆: 结构性质保证对数高度, 堆序性质保证根是全局最小元, `findMin()` 是 $O(1)$ 。

立刻澄清: 堆序只约束“父子”, 兄弟之间没有大小关系。堆因此不是有序数组, 不能二分查找, 按值查找只能全扫 $O(n)$ 。堆只承诺最小元在根, 是“弱有序”结构, 与第4章 BST 的强有序对照。

6.2.3 数组表示:0 基与 1 基换算

把完全二叉树按层序放进数组, 父子关系退化为下标运算。这里出现全系列最重要的“习惯分歧”: Python 的 `heapq` 用 0 基——根在下标 0; C++ 教材(Weiss 原书)用 1 基——根在下标 1, 下标 0 闲置。两套公式:

含义	0 基(Python 用)	1 基(C++ 教材习惯)
根	下标 0	下标 1
结点 i 的左子	$2i+1$	$2i$
结点 i 的右子	$2i+2$	$2i+1$
结点 i 的父	$(i-1)//2$ (整数除法)	$i/2$ (整数除法)
最后一个非叶结点	$n//2 - 1$	$n/2$
下标 0 是否闲置	否, 下标 0 就是根	是, <code>a[0]</code> 闲置

验证: 下标 5 的结点, 0 基下左子 11、右子 12、父 2; 1 基下左子 10、右子 11、父 2。父相同只是巧合——下标 4: 0 基父 1, 1 基父 2。

⚠ 误区 1:把 1 基公式抄进 0 基代码(或反之)

两种下标体系并存是本章第一大坑。0 基堆里左子必须写 $2*i+1$ 、父必须写 $(i-1)//2$ ——抄成 1 基的 $2*i$ 、 $i//2$ 会取到错位置;建堆时 1 基从 $n//2$ 开始,0 基必须从 $n//2-1$ 开始,差一个结点。解决之道:写代码前先声明"用几基"并写进注释,写完用"验证堆序"函数检查 (§6.3.3)。

示例 6-1 二叉堆类骨架(0 基,list 作底层)

```
# 0 基堆:根在下标 0,左子 2i+1、右子 2i+2、父 (i-1)//2
class MinHeap:
    def __init__(self):
        self.a = []          # 底层就是 list,无需占位

    def __len__(self):
        return len(self.a)

    def top(self):
        return self.a[0]     # 根即最小元 0(1)

    def check_heap(self):
        """调试辅助:验证堆序(见 §6.3.3)"""
        for i in range(len(self.a)):
            for c in (2 * i + 1, 2 * i + 2):
                if c < len(self.a) and self.a[i] > self.a[c]:
                    return False
        return True
```

§6.3 插入与删除最小:上滤与下滤

6.3.1 插入:上滤(percolateUp)

插入策略是"先保结构,再修堆序": ① 新元素追加到末尾 (完全二叉树只在末尾留空位,结构自动保持); ② 沿根路径向上冒泡——与父比较,小于父则交换,直到父 $\leq$ 自己或到根。这叫**上滤(percolate up)**,路径最长即树高,insert 为 $O(\log n)$; 上滤只看一个父,正是堆"弱有序"的便利。

示例 6-2 insert:上滤

```
def push(self, x):
    self.a.append(x)          # 追加到末尾:保持完全二叉树
    i = len(self.a) - 1      # 新元素下标
    while i > 0:              # 向上冒泡(上滤)
        p = (i - 1) // 2      # 父结点(0 基公式)
        if self.a[p] <= self.a[i]:
            break              # 父 <= 子,堆序成立,停
        self.a[p], self.a[i] = self.a[i], self.a[p]  # 与父交换
        i = p
```

6.3.2 删除最小:下滤(percolateDown)

删除根会留下空洞,朴素前移会破坏完全二叉树或付出 $O(n)$ 搬移。经典套路: ① 记下根(它就是答案); ② 把**最后一个元素搬到根**(末尾是唯一可合法删除的空位,结构自动保持); ③ 沿"叶方向"向下冒泡——与孩子中**较小者**比较,大于它则交换,直到 $\leq$ 两个孩子或到叶。这叫**下滤(percolate down)**,同样 $O(\log n)$ 。

两个细节:必须与"较小的"孩子交换(与较大的换会违反堆序);先检查右子是否存在(下标 $2i+2$ 是否在界内),只有左子也合法。

示例 6-3 deleteMin:末元素搬根再下滤

```
def _sift_down(self, i):
    """下滤:把下标 i 的元素向下沉(deleteMin 与建堆共用)"""
    n = len(self.a)
    while True:
        smallest = i
        for c in (2 * i + 1, 2 * i + 2):      # 左子、右子(0 基)
            if c < n and self.a[c] < self.a[smallest]:
                smallest = c                # 选"较小"的孩子
        if smallest == i:
            break                          # 两个孩子都不小于自己
        self.a[i], self.a[smallest] = self.a[smallest], self.a[i]
        i = smallest

def pop(self):
    """删除最小元:最后一个元素搬到根,再下滤"""
    top = self.a[0]                        # 记下根(答案)
    last = self.a.pop()                   # 末元素搬根:保持完全二叉树
    if self.a:                             # 只剩 1 个元素则免
        self.a[0] = last
        self._sift_down(0)
    return top
```

例题 6-1 手工走一遍 insert 与 deleteMin

设 0 基堆初始为 `[13, 21, 16, 24, 31, 19, 68, 65, 26, 32]`。**insert(14)**:追加到下标 10,沿 $10 \rightarrow 4 \rightarrow 1 \rightarrow 0$ 上滤: $31 > 14$ 换、 $21 > 14$ 换、 $13 \leq 14$ 停,得 `[13, 14, 16, 24, 21, 19, 68, 65, 26, 32, 31]`。**deleteMin()**:取走 13,末元素 31 搬根,依次换过 14、21、26,得 `[14, 21, 16, 24, 26, 19, 68, 65, 31, 32]`,最小元 14。

6.3.3 完整接口与调试技巧

完整堆还应有 `top()`、`__len__` (配合 `len(h)`) 等接口;若要"按插入顺序稳定",可在元素里加序号比较(见 §6.7 的 (频次, 词) 元组)。

调试技巧:写一个"验证堆序"的断言函数

堆的 bug 极具隐蔽性:上滤少换一层、下滤选错孩子,结果仍"接近正确"。随手加 `check_heap()` (示例 6-1 已给出):遍历所有结点,断言父 $\leq$ 每个存在的孩子,每次 push/pop 后调用,把可疑行为扼杀在摇篮里。

§6.4 建堆与其余操作

6.4.1 两种建堆方式

建堆有两种做法。① **自顶向下**:空堆逐个 insert,每次 $O(\log n)$,共 $O(n \log n)$ 。② **自底向上(下滤建堆)**: n 个元素按任意顺序放进数组(天然构成完全二叉树,只是堆序可能被违反),从最后非叶结点($n//2-1$)往前逐个 percolateDown 到根,总代价 $O(n)$ 。"一次性给全所有元素"的场景都用下滤建堆。

示例 6-4 下滤建堆: $O(n)$

```
@classmethod
def build_from(cls, items):
    """下滤建堆  $O(n)$ : 从最后一个非叶结点( $n//2-1$ )开始逐个下滤到根"""
    h = cls()
    h.a = list(items)
    for i in range(len(h.a) // 2 - 1, -1, -1):
        h._sift_down(i)
    return h
```

例题 6-2 手工走一遍下滤建堆

输入 [150, 80, 40, 30, 10, 70, 110, 100, 20, 90, 60, 50, 120, 140, 130] ($n = 15$, 最后非叶 = 下标 6)。
 从 6 往前: 6(110) ≤ 130 停; 5(70) 与 50 换; 4(10) ≤ 90、60 停; 3(30) 与 20 换; 2(40) ≤ 50、110 停; 1(80) 与 10 换、再与 60 换; 0(150) 依次与 10、20、30 换。最终 [10, 20, 40, 30, 60, 50, 110, 100, 80, 90, 70, 150, 120, 140, 130]。

6.4.2 为什么下滤建堆是 $O(n)$: 一句话推导

关键观察: 高度为 h 的结点约有 $n/2^{h+1}$ 个, 每个下滤至多走 h 步, 总代价 $\sum_h h \cdot n/2^{h+1}$ 是“几何级数收尾”的求和, 结果 $O(n)$ 。直觉: 矮的结点数量多但只需下滤几步, 高的结点上滤远但数量极少; 逐个插入的代价全集中在“深叶子上滤到根”的长路径。严格证明在第11章。

6.4.3 decreaseKey、delete 与 merge

- **decreaseKey(p, delta)**: 键变小只能向上走——直接上滤, $O(\log n)$ 。前提是知道 p 的位置(堆不提供查找); 经典用途: Dijkstra 松弛(第9章)。
- **delete(任意位置 p)**: 把 $a[p]$ 换成最小值(根的副本), 上滤到根, 再 deleteMin——两步都 $O(\log n)$ 。
- **merge(合并两个堆)**: 二叉堆的短板——必须把两个数组拷到一起再建堆, $O(n)$ 。频繁合并堆的应用, 二叉堆不够用, 这正是 §6.5 堆家族的动机。

示例 6-5 decreaseKey 与 delete(任意位置)

```
def decrease_key(self, i, delta):
    """键变小只会"向上"走: 减键后上滤  $O(\log n)$ """
    self.a[i] -= delta
    while i > 0:
        p = (i - 1) // 2
        if self.a[p] <= self.a[i]:
            break
        self.a[p], self.a[i] = self.a[i], self.a[p]
        i = p

def remove_at(self, i):
    """删除任意位置: 盖成最小值上滤到根, 再 pop  $O(\log n)$ """
    self.a[i] = self.a[0]          # 根是最小值, 盖上去不破坏堆序
    while i > 0:                  # 一路换到根
        p = (i - 1) // 2
        self.a[p], self.a[i] = self.a[i], self.a[p]
        i = p
    self.pop()                   # 现在根就是原来的 a[i]
```


§6.5 其他堆:d-堆、左式堆、斜堆与二项队列

6.5.1 为什么不止一种堆

二叉堆已做到 insert/deleteMin 均 $O(\log n)$ 、 findMin $O(1)$,为何还要别的堆?因为**操作组合不同**:插入远多于删除的系统希望 insert 更快;频繁合并堆的系统受不了 merge $O(n)$ 。没有一种堆能同时最优,每个成员都在特定操作上让步、换取另一操作的改进。

6.5.2 d-堆:多叉,降低高度

d-堆 是二叉堆的直接推广:每个结点最多 d 个孩子(二叉堆即 $d = 2$)。高度从 $\log_2 n$ 降到 $\log_d n$, insert (只与一个父比较)变快为 $O(\log_d n)$;但 deleteMin 要在 d 个孩子里找最小,变慢为 $O(d \log_d n)$ 。典型场景: insert 远多于 deleteMin 的离散事件模拟,常用 $d = 4$ 或 5 。

6.5.3 左式堆:为 merge 而生

二叉堆 merge $O(n)$ 的根源是"完全二叉树"锁死了形状。**左式堆(leftist heap)** 主动放弃完全性,改用新约束:对每个结点,左子的零路径长(NPL)不小于右子的——即"左偏"。**零路径长(null path length)** 是到最近叶子的最短路径长度(空结点为 -1)。它保证最右路径很短, merge 递归沿最右路径合并, $O(\log n)$; insert/deleteMin 都可用 merge 实现。

6.5.4 斜堆:左式堆的摊还简化版

斜堆(skew heap) 是左式堆的极简主义版本:不维护 NPL, merge 后**无条件交换左右子树**。实现更短,但单次可能退化 $O(n)$,整体摊还 $O(\log n)$ (第11章证明)。它与左式堆的关系,正如伸展树之于 AVL 树。

6.5.5 二项队列:森林

二项队列(binomial queue) 的思路完全不同:不是一棵树,而是一组"二项树"组成的**森林**。第 k 棵二项树大小恰为 2^k ,每种大小最多一棵—— n 个元素按二进制位决定有哪些树(如 $n = 13 = 1101$,有 8 、 4 、 1 三棵)。插入像二进制加法"进位"合并同大小树,摊还 $O(1)$; merge 与 deleteMin 均 $O(\log n)$ 。

堆类型	动机	insert	deleteMin	merge
二叉堆	最常用,数组存储、缓存友好	$O(\log n)$	$O(\log n)$	$O(n)$
d-堆	插入多于删除,降高度	$O(\log_d n)$	$O(d \log_d n)$	$O(n)$
左式堆	高效 merge,左偏约束	$O(\log n)$	$O(\log n)$	$O(\log n)$
斜堆	左式堆的摊还简化版	摊还 $O(\log n)$	摊还 $O(\log n)$	摊还 $O(\log n)$
二项队列	森林结构,插入摊还 $O(1)$	摊还 $O(1)$	$O(\log n)$	$O(\log n)$

§6.6 标准库:heapq(默认小顶堆与负号技巧)

6.6.1 四个函数:heapify、heappush、heappop、heapreplace

`heapq` 是标准库的堆工具箱,与 C++ 的 `std::priority_queue` 位置相同,但设计哲学完全不同:它**不提供堆对象**,只有一组操作**普通 list** 的函数——你拿一个 list 调 `heappush`、`heappop`,list 就地变成堆。最大的特点:它默认是"小顶堆",与 `priority_queue` 默认大顶堆正好相反。

- `heapify(lst)`:把任意 list 原地建成小顶堆, $O(n)$ ——即下滤建堆;

- `heappush(lst, x)` :插入并上滤, $O(\log n)$;
- `heappop(lst)` :弹出并返回堆顶(最小元), 末元素搬根再下滤, $O(\log n)$;
- `heapreplace(lst, x)` :先弹出堆顶再压入 x (一步完成), 等价于 `heappop + heappush` 但更快——top-k 淘汰的标准写法。

示例 6-6 `heapq` 四函数与负号技巧

```
import heapq

lst = [3, 1, 9, 5]
heapq.heapify(lst)           # ① 原地建小顶堆  $O(n)$ 
# lst == [1, 3, 9, 5](堆序成立, 兄弟间无序)

heapq.heappush(lst, 2)       # ② 插入并上滤: lst == [1, 2, 9, 5, 3]
m = heapq.heappop(lst)       # ③ 弹出最小元: m == 1
heapq.heappush(lst, 0)
heapq.heapreplace(lst, 7)    # ④ 弹出堆顶(0)再压入 7

# 负号技巧: heapq 没有大顶堆, 存负值即得"大顶堆"
big = []
for x in [3, 1, 9, 5]:
    heapq.heappush(big, -x)   # 存 -x: 最小堆里最小的是 -9
max_val = -heapq.heappop(big) # 取回时再取负: 9
```

 误区 2: 负号技巧只取一半(忘负或忘取负)

`heapq` 没有 `greater<T>` 那样的比较器参数, 大顶堆只能靠存负值: 压入 `-x`, 弹出后 `-heapq.heappop(...)`。新手要么压入时忘取负(堆里全是负数, 弹出来全是负的), 要么弹出时忘取负(拿到 `-9` 而不是 `9`)。口诀: 负进负出。另一个坑: 取负要求元素可被 `abs` -取负——字符串、元组整体取负会 `TypeError`, 只能对数值字段取负(见 §6.6.2)。

6.6.2 自定义优先级: 元组天然可比较

`priority_queue` 靠模板参数传比较器; `heapq` 没有比较器参数, 靠**元素本身**可比: 存 (频次, 词) 元组, Python 自动先比频次、平局比词序, 零配置。求"最大的 k 个"时更常用 (-频次, 词) 或 (频次, 词) 配合小顶堆淘汰 (§6.7)。

C++ 的 <code>std::priority_queue</code>	Python 的 <code>heapq</code>
封装成类, 构造时传比较器	无堆对象, 四函数直接操作普通 list
默认大顶堆, 最小堆要写 <code>greater</code>	默认小顶堆, 大顶堆用负号技巧
<code>pop</code> 不返回元素, 先 <code>top</code> 再 <code>pop</code>	<code>heappop</code> 直接返回被弹元素
批量建堆: 构造时拷入容器	<code>heapify</code> 原地建堆, 零拷贝
自定义比较靠比较器模板参数	自定义比较靠元素本身(元组/负号)

 工程建议: 什么时候直接上 `heapq`

工程中"需要动态维护最值"的场景一律优先 `heapq`: 任务调度、Top-K、贪心(第10章哈夫曼)、Dijkstra(第9章)。手写堆的用武之地: 需要 `decreaseKey`(标准库没有)、批量建堆后长期复用、或考试要求展示原理。只"一次性排序"用 `sorted()` (常数更小); "边插入边取最值"才用堆。

§6.7 手写堆与标准库并排:top-k 实战

6.7.1 问题:求最大的 k 个(top-k)

Top-K 问题:给定 n 个元素,求最大的 k 个(排行榜、热门词)。朴素做法全部排序是 $O(n \log n)$;堆做法 $O(n \log k)$, k 远小于 n 时近乎线性。核心技巧叫**小顶堆淘汰法**:维护"当前最大的 k 个"的小顶堆——堆顶恰是第 k 大,即淘汰"门槛";新元素大于堆顶才替换。

为什么用"小顶堆"而非"大顶堆"? 大顶堆的堆顶是全局最大,淘汰后不知谁替补;小顶堆的堆顶是"最小的入选者",谁比它大谁挤掉它,淘汰逻辑一行搞定。

C 的朴素做法(qsort 全排序)	Python 的堆做法(heapq)
qsort 把 n 个元素全部排序, $O(n \log n)$	大小为 k 的小顶堆, $O(n \log k)$
结果数组占 n 个位置	堆只占 k 个位置,可流式处理
新数据到达必须重新 qsort 一遍	heappush/heapreplace 动态维护,每次 $O(\log k)$
比较函数用函数指针,写在调用处之外	元组天然可比较,零配置

⚠ 误区 3:top-k 时"无脑入堆",忘了先与堆顶比较

堆满后仍直接 heappush,堆会涨过 k ;或"先 push 再 pop 堆顶",把刚进来的大元素又弹走——淘汰条件写反。正确顺序只有一种:先与堆顶比较,大于门槛才 pop 再 push,且 pop 必须在 push 之前。用 heapreplace 一步完成弹旧压新,从机制上杜绝顺序错误。

6.7.2 手写版与标准库版并排

同一任务——给一组(频次,词)求频次最高的 k 个——分别用手写堆与 heapq 实现。两份代码逻辑结构一字不差,正是"手写理解原理、标准库交付工程"的教科书式对照。

示例 6-7 手写堆版 top-k(复用示例 6-1~6-3 的 MinHeap)

```
# items: (频次, 词) 列表;结果: 按频次降序输出最大的 k 个
def topk_heap(items, k):
    h = MinHeap()
    for t in items:
        # t 是 (频次, 词) 元组
        if len(h) < k:
            h.push(t)
            # 堆未满:直接入堆
        elif t > h.top():
            # 高于"第 k 大门槛"才替换
            h.pop()
            h.push(t)
    res = []
    while len(h) > 0:
        res.append(h.pop())
    return list(reversed(res))
# 小顶堆弹出是升序,反转为降序
```

示例 6-8 heapq 版 top-k:逻辑一字不差

```
def topk_heapq(items, k):
    """同一任务用 heapq.heappush/heappreplace 换掉手写调用"""
    h = []
    for t in items:
        if len(h) < k:
            heapq.heappush(h, t)
        elif t > h[0]:
            # h[0] 就是小顶堆的堆顶
            # 弹旧压新: 从机制上杜绝顺序错误
            heapq.heapreplace(h, t)
    return sorted(h, reverse=True)
    # 堆内即答案, 排序只为了降序展示
```

6.7.3 复杂度分析与堆排序预告

每个元素至多一次比较 + 一次替换 ($O(\log k)$), 总代价 $O(n \log k)$; $k = n$ 时退化为 $O(n \log n)$ 。优势在**内存**: 堆只占 k 个位置, 几十 GB 的日志放不进内存, 却能流式处理。

预告第7章: 堆排序直接复用本章的二叉堆——建堆 $O(n)$, 连续 `deleteMin` n 次, 弹出元素天然有序, 总代价 $O(n \log n)$, 与插入、希尔、归并、快排逐一对比。

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
insert	二叉堆	$O(\log n)$	$O(\log n)$
deleteMin / deleteMax	二叉堆	$O(\log n)$	$O(\log n)$
findMin	二叉堆	$O(1)$	$O(1)$
buildHeap(下滤建堆)	二叉堆	$O(n)$	$O(n)$
buildHeap(逐个插入)	二叉堆	$O(n \log n)$	$O(n \log n)$
decreaseKey	二叉堆	$O(\log n)$	$O(\log n)$
delete(任意元素)	二叉堆	$O(\log n)$	$O(\log n)$
merge	二叉堆	$O(n)$	$O(n)$
按值查找 find	二叉堆	$O(n)$	$O(n)$
insert	d-堆	$O(\log_d n)$	$O(\log_d n)$
deleteMin	d-堆	$O(d \log_d n)$	$O(d \log_d n)$
merge	左式堆	$O(\log n)$	$O(\log n)$
merge	斜堆	摊还 $O(\log n)$	$O(n)$
insert	二项队列	摊还 $O(1)$	$O(\log n)$
deleteMin	二项队列	$O(\log n)$	$O(\log n)$
merge	二项队列	$O(\log n)$	$O(\log n)$
top-k(大小为 k 的小顶堆)	二叉堆 + 一次遍历	$O(n \log k)$	$O(n \log k)$

最小必会清单

- 能说出优先队列 ADT 的三个操作与普通队列的本质区别
- 能说出二叉堆的两条性质,并画出完全二叉树的数组填充
- 能默写 0 基公式(左子 $2i+1$ 、右子 $2i+2$ 、父 $(i-1)//2$ 、最后非叶 $n//2-1$),并与 1 基公式换算
- 能默写 insert(上滤)与 deleteMin(末元素搬根再下滤)的代码
- 能解释"删除最小为什么是搬最后一个元素",以及"下滤为什么必须选较小的孩子"
- 能独立写出下滤建堆,并解释两种建堆方式的复杂度差异
- 能说出 decreaseKey、delete、merge 的代价与场景
- 能一句话说出 d-堆、左式堆、斜堆、二项队列的动机与差别
- 能熟练写出 heapify/heappush/heappop/heapreplace 的用法,并解释负号技巧
- 能独立用"大小为 k 的小顶堆"解决 top-k,并解释为什么不用大顶堆

自测题

Q 1. 二叉堆为什么必须是完全二叉树?如果允许任意形状的二叉树,会损失什么?

✓ 参考答案

完全性保证高度恒为 $\log_2 n$ 与数组连续存储(无指针、下标算父子)。若形状任意,最坏退化成链,高度与操作都变 $O(n)$,还得用指针。

Q 2. 0 基堆中下标 i 的结点的父、左子、右子分别是什么?1 基堆呢?试各举一例。

✓ 参考答案

0 基:父 $(i-1)//2$ 、左子 $2i+1$ 、右子 $2i+2$ (如 $i=5$:父 2、左 11、右 12);1 基:父 $i/2$ 、左子 $2i$ 、右子 $2i+1$ (如 $i=5$:父 2、左 10、右 11)。“父为 2”只是巧合,不可混用。

Q 3. 在最小堆中查找某个值的复杂度是多少?为什么不能像 BST 那样二分?

✓ 参考答案

$O(n)$ 。堆序只约束父子关系,兄弟之间没有大小关系,无法一次比较排除一半子树;堆也不是有序数组,连续存储帮不上二分。堆只承诺最小元在根。

Q 4. 把 15 个元素下滤建堆,最后一个下滤的结点是哪个?为什么不是第一个?

✓ 参考答案

最后下滤的是根(下标 0)。建堆从最后非叶结点($n//2-1$)往前处理,先让下面子树各自成堆,上层下滤时才能保证孩子子树已是堆、只需比较两个孩子。根最后处理,一次下滤把最小值带到根。

Q 5. deleteMin 为什么要把"最后一个元素"搬到根,而不是把后面元素整体前移?

✅ 参考答案

整体前移会留下空隙(破坏结构性质)或付出 $O(n)$ 搬移。最后一个元素是唯一"删除后数组仍连续"的位置:先搬到根保持结构,再下滤修复堆序,两步各 $O(\log n)$ 。

Q 6. 求 top-k 为什么用大小为 k 的"小顶堆"?用"大顶堆"行吗?

✅ 参考答案

小顶堆的堆顶是"当前入选者里最小的"(第 k 大门槛),新元素与堆顶比较即可决定淘汰谁。大顶堆的堆顶是全局最大,被淘汰后无法 $O(1)$ 判断谁替补,只能维护更多信息或退化成排序。求"最小的 k 个"改用大小为 k 的大顶堆。

章末实验题:Top-K 高频词(堆实现)

📄 实验 6:Top-K 高频词(手写堆 + 标准库双实现)

实验目标:流式读入英文文本,先用**手写二叉堆**求频次最高的 k 个词,再用 `heapq` 重做并验证两版结果一致。

实验要求:

- 任务 1:实现二叉堆类(insert/deleteMin/下滤建堆,0 基,注释中说明下标公式)(知识点:\$6.2-\$6.4 上滤/下滤/建堆)
- 任务 2:流式读入文本,切分出单词并维护"词频"字典(知识点:\$6.1 + 第5章散列)
- 任务 3:用"大小为 k 的小顶堆"求频次最高的 k 个词,淘汰技巧:堆满时频次高于堆顶才替换(知识点:\$6.7 top-k)
- 任务 4:用 `heapq` (`heappush`/`heapreplace`)重做任务 3,对比两版结果一致(知识点:\$6.6 标准库)
- 任务 5:处理边界: $k \leq 0$ 、文本为空、k 大于词数时给出合理输出而非崩溃(知识点:\$6.3)

实验环境:Python 3。运行: `python lab6.py k < 文本文件`。

第一步 写 `MinHeap` 类(0 基),用例题 6-1 的序列手工验证 `insert` 与 `deleteMin`,再写 `top-k` 函数

第二步 切分用 `re.findall(r"[a-zA-Z']+", text.lower())` 一行完成

第三步 元素用 (频次, 词) 元组:先比频次、平局比词序

第四步 `heapq` 版用 `heapreplace` 一步完成弹旧压新;结果排序用 `sorted(reverse=True)`

第五步 运行样例: `python lab6.py 3 < sample.txt`,再试 $k = 0$ 与空文件

✅ 参考答案(完整可运行程序,约 120 行,分三段)

```

# lab6.py — 第一段:MinHeap(0 基)
# 运行: python lab6.py k < 文本文件
import heapq
import re
import sys

class MinHeap:
    """任务 1:手写二叉堆(0 基:根在下标 0,左子 2i+1、右子 2i+2、父 (i-1)//2)。
    元素为 (频次, 词) 元组:频次小者优先级高,平局按词序比较。"""

    def __init__(self):
        self.a = []                # 底层就是普通 list,无需占位

    def __len__(self):
        return len(self.a)

    def _sift_up(self, i):
        """上滤:从下标 i 向上冒泡,直到父 <= 自己。"""
        while i > 0:
            p = (i - 1) // 2        # 0 基父结点公式
            if self.a[p] <= self.a[i]:
                break
            self.a[p], self.a[i] = self.a[i], self.a[p]
            i = p

    def _sift_down(self, i):
        """下滤:从下标 i 向下沉,始终与较小的孩子交换。"""
        n = len(self.a)
        while True:
            smallest = i
            for c in (2 * i + 1, 2 * i + 2): # 左子、右子(0 基)
                if c < n and self.a[c] < self.a[smallest]:
                    smallest = c
            if smallest == i:
                break
            self.a[i], self.a[smallest] = self.a[smallest], self.a[i]
            i = smallest

    def push(self, x):
        self.a.append(x)
        self._sift_up(len(self.a) - 1)

    def pop(self):
        """删除最小元:最后一个元素搬到根再下滤($6.3 套路)。"""
        top = self.a[0]
        last = self.a.pop()
        if self.a:
            self.a[0] = last
            self._sift_down(0)
        return top

    def top(self):
        return self.a[0]

```


lab6.py(第一段补充:下滤建堆 build_from)

```
@classmethod
def build_from(cls, items):
    """任务 1 补充:下滤建堆  $O(n)$ —从最后一个非叶结点往上逐个下滤。"""
    h = cls()
    h.a = list(items)
    for i in range(len(h.a) // 2 - 1, -1, -1):
        h._sift_down(i)
    return h
```

lab6.py(第二段:词频统计与手写堆 top-k)

```
def count_words(text):
    """任务 2:流式切词并统计词频,维护"词频"字典。"""
    freq = {}
    for w in re.findall(r"[a-zA-Z']+", text.lower()):
        freq[w] = freq.get(w, 0) + 1
    return freq

def topk_heap(items, k):
    """任务 3:手写小顶堆求频次最高的 k 个(小顶堆淘汰技巧)。"""
    h = MinHeap()
    for t in items:
        # t 是 (频次, 词) 元组
        if len(h) < k:
            h.push(t)
            # 堆未满:直接入堆
        elif t > h.top():
            # 高于"第 k 大门槛"才替换
            h.pop()
            h.push(t)
    res = []
    while len(h) > 0:
        res.append(h.pop())
    return list(reversed(res))
    # 小顶堆弹出是升序,反转为降序
```

lab6.py(第三段:heapq 版与主程序)

```

def topk_heapq(items, k):
    """任务 4:标准库 heapq 重做,逻辑与手写版一字不差。"""
    h = []
    for t in items:
        if len(h) < k:
            heapq.heappush(h, t)
        elif t > h[0]:                # h[0] 就是小顶堆的堆顶
            heapq.heapreplace(h, t)   # 等价于 heappop + heappush
    return sorted(h, reverse=True)

def main():
    # 用法: python lab6.py k < 文本文件
    if len(sys.argv) < 2:
        print("用法: python lab6.py k < 文本文件")
        return
    k = int(sys.argv[1])
    text = sys.stdin.read()
    freq = count_words(text)         # 任务 2:词频字典
    items = [(c, w) for w, c in freq.items()]  # (频次, 词)

    # 任务 5:边界处理—k <= 0 或文本为空
    if k <= 0 or not items:
        print(f"k = {k}, 词数 = {len(items)}:边界情形,答案为空。")
        return

    r1 = topk_heap(items, k)
    r2 = topk_heapq(items, k)
    print(f"词数 = {len(items)}, k = {k}")
    print("手写堆 :", [(w, c) for c, w in r1])
    print("heapq :", [(w, c) for c, w in r2])
    print("结果一致:", r1 == r2)

if __name__ == "__main__":
    main()

```

示例输入与输出(真实运行;sample.txt 与 empty.txt 为测试文件):

```

$ python lab6.py 3 < sample.txt
词数 = 34, k = 3
手写堆 : [('to', 6), ('the', 3), ('or', 2)]
heapq  : [('to', 6), ('the', 3), ('or', 2)]
结果一致: True

$ python lab6.py 5 < empty.txt
k = 5, 词数 = 0:边界情形,答案为空。

```

设计说明:① 0 基公式写进类注释,与 C++ 版 1 基实现互为对照;② 元素用 (频次, 词) 元组,平局比词序,结果可复现;③ 两版结果一致即互相验证;④ 词频统计用 `re.findall` + `dict`(第5章),与堆协同完成统计 → Top-K 流水线;⑤ 边界显式处理,避免空堆访问。

下一章预告:第7章 排序——堆排序将直接使用本章的二叉堆:下滤建堆 $O(n)$,再连续 deleteMin n 次得到有序序列,总代价 $O(n \log n)$;插入、希尔、归并、快排逐一登场。